Question 1

You just joined a company as a junior developer. Your team lead asks you to go to the project folder called `myapp`, then go inside a subfolder called `src`, and list all the files there. You open the terminal but you don't know where you are. What commands will you run, step by step?

Answer:

pwd <-- to find the current directory -->

cd myapp/ <-- to change present directory to 'myapp' -->

cd src/ <-- to step inside the 'src' directory -->

ls <-- to list all the files inside the directory -->

Question 2

You are working on a project and your manager asks you to create a new folder called `reports` inside your project directory, then create an empty file called `summary.txt` inside it. How will you do this using the terminal?

Answer:

We can achieve this by running the following commands in sequence:

1. `mkdir reports` → This creates the new directory named `reports`.

2. `cd reports` → This moves you into the newly created `reports` folder.

3. `touch summary.txt` → This creates an empty text file named `summary.txt` inside the current folder.

Question 3

You wrote some important notes in a file called `notes.txt`. Your senior developer asks you to check what is written in that file quickly, without opening any editor. What command will you use and why?

Answer:

Command: `cat notes.txt`

Reason: The `cat` command reads the content of a file and prints it directly onto the terminal screen. This is the fastest method because it doesn't launch an interactive text editor (like Nano or Vim), saving time and keeping you right in your command-line workflow.

Question 4

You are setting up a web server on a Linux machine. You run your server script but it says 'Permission denied'. Your senior tells you the file needs to be executable. What will you do to fix this?

Answer:

We need to change the file permissions using the `chmod` command. Assuming your script is named `script.sh`,

run:

chmod +x script.sh

Our file is now executable.

Question 5

You have a large log file called `app.log` and you need to find all lines that contain the word 'error'. The file has thousands of lines and you cannot read it manually. What command will you use?

Answer:

Command: `grep "error" app.log`

Reason: The `grep` command searches through a specified file for a matching text string and prints out every line containing that string. To make it case-insensitive (catching "Error" or "ERROR" as well), you can add the `-i` flag: `grep -i "error" app.log`.

Question 6

You are running a command that generates a long list of files. You want to see only the files whose names contain the word 'config'. How will you combine two commands to do this in one line?

Answer:

By combining the listing command (`ls`) and the search command (`grep`) using a “pipe (`|`)”.

Command: `ls | grep "config"`

The pipe character (`|`) takes the output of the first command (`ls`) and passes it directly as input to the second command (`grep`). `grep` then filters that list to remove anything that doesn't contain the phrase "config".

Question 7

A company's website is not working. The developer checks the HTML file and sees there is no DOCTYPE declaration at the top and the content is placed directly without any head or body tags. Why could this be a problem?

Answer:

This causes structural degradation and rendering errors for several reasons:

Quirks Mode: Without `<!DOCTYPE html>`, modern browsers will drop into "Quirks Mode," rendering the site using legacy, non-standard layout rules that break modern CSS layouts.

Malformed Document Tree (DOM): Missing `<head>` and `<body>` tags force browsers to guess where metadata stops and visible content begins. This can cause scripts to execute prematurely, break external stylesheet links, and disrupt JavaScript DOM manipulation.

SEO and Accessibility Issues: Search engines and screen-reading software rely on proper semantic structures to index webpages and read content effectively.

Question 8

You are building an online store page for a shop in your city. The page has a product listing, a contact form, a header with the shop name, and a footer with address. Your senior says you should use semantic HTML tags. Which tags will you use for each section and why?

Answer:

Shop Name Section: Use `<header>`. It clearly defines the introductory container or navigation hub of the document.

Product Listing Section: Use `<main>` to encapsulate the core unique content of the page, and fill it with `<section>` to group the catalog, using individual `<article>` tags for each unique product item. This tells browsers and screen readers that each item is a distinct, self-contained entity.

Contact Form Section: Use `<section>` containing a `<form>` tag. The `<form>` tag is structurally crucial because it defines interactive controls to collect user data.

Address Section: Use `<footer>`. It designates the bottom of the page layout, and nesting an `<address>` tag inside it is semantically optimal for contact coordinates.

Question 9

What is the difference between `>` and `>>` in the terminal? Give one example of when you would use each.

Answer:

‘>’ (Overwrite Redirect): Redirects standard output to a file. If the target file already exists, it completely erases (overwrites) its current contents. If the file doesn't exist, it creates a new one.

Example: `echo "Hello World" > greetings.txt` (Creates or completely overwrites `greetings.txt` to contain only "Hello World").

‘>>’ (Append Redirect): Redirects standard output to a file but appends the new text to the very end of the file without deleting its existing contents.

Example: `echo "New log entry" >> app.log` (Safely logs data to the bottom of your running logs file without wiping out history).

Question 10

What is the difference between `rm` and `rm -r`? Why should you be careful when using `rm -r`?

Answer:

rm: Used to delete “individual files”. It will throw an error if you attempt to use it on a directory.

‘rm -r’: The ‘-r’ stands for “recursive”. This command deletes a directory along with all of its subdirectories, folders, and inner files.

Why be careful: The command line does not have a "Trash Bin" or an "Undo" feature. Once you hit enter on `rm -r`, everything inside that target tree is permanently deleted instantly. Accidentally pointing it to the wrong directory can erase critical systems or wipe weeks of work in a split second.

Question 11

What is the difference between semantic HTML tags like `section` and `article` versus a generic `div` tag? Why does it matter?

Answer:

The Difference: A `<div>` tag is a completely non-semantic container; it carries zero meaning about the text or elements inside it and is used purely for layout/styling. Conversely, semantic tags like `<section>` (representing a thematic grouping of content) and `<article>` (representing a piece of content that can stand independently on its own, like a blog post) communicate exactly what kind of content they hold.

It matters because:

1. Accessibility: Screen readers rely on semantic anchors to help visually impaired users skip directly to primary navigation blocks or articles.

2. SEO Optimisation: Search engine crawlers evaluate semantic code hierarchies to determine which keywords and text sections are most valuable on your page, resulting in better search ranking.

3. Maintainability: Readable, semantic structures are significantly easier for development teams to scan and maintain than nested walls of confusing "div soup."

Question 12

Why are forms important in web development? Name three input types used in HTML forms and explain what each is used for.

Answer:

Importance: Forms are the primary gateway for two-way user interaction on the web. Without forms, websites would be entirely read-only; forms allow user management systems to accept login credentials, process e-commerce payments, register accounts, and collect user search queries or feedback.

Three Input Types:

1. `<input type="text">` — Renders a simple, single-line text field used for entering basic textual data like names, usernames, or addresses.

2. `<input type="email">` — Specifically structured for email addresses. It includes automatic validation rules built into browsers that reject entries missing an `@` symbol or domain configuration.

3. `<input type="password">` — Renders a secure data field where the characters typed are masked automatically (appearing as dots or asterisks) to prevent shoulder-surfing and protect user privacy.